Alma Mater Studiorum – University of Bologna

Master's Degree in Computer Science and Engineering
Major in Intelligent Embedded Systems

Vehicle Fleet Predictive Maintenance: Domain-Driven Design and DevOps

Software Process Engineering Project

Course: Software Process Engineering (Codice B0011)
Repository: `https://github.com/Dias266/Multi-Agent-System-for-Autonomous-Vehicle-Predictive-Maintenance`

Students:

Danial Khayatian – danial.khayatian@studio.unibo.it
Dias Katrenov – dias.katrenov@studio.unibo.it
Mary Anne Selirio – maryanne.selirio@studio.unibo.it

Academic Year 2025/2026 · October 2026

Abstract

This report describes how Domain-Driven Design (DDD) and DevOps were applied to an existing Jason (AgentSpeak) multi-agent system for the predictive maintenance of a vehicle fleet. On the **DDD** side, the domain was analysed through knowledge crunching, a ubiquitous language, use cases and five bounded contexts with a context map; the Trust & Service Ledger and Maintenance Prediction contexts are modelled tactically and run in production code, behind ports and adapters in a Clean Architecture. Service records written by the running agents are stored in PostgreSQL, and the rule that a record is decided exactly once is enforced by both the aggregate and the database. On the **DevOps** side, every push runs a CircleCI pipeline that checks the version and lints, tests and measures coverage on the JVM, Python and Node.js. On `main` and on release tags, a delivery job builds five images, smoke-tests the whole system (including PostgreSQL and an MQTT round trip through the real broker) and publishes multi-architecture images to the GitHub Container Registry. The repository is MIT-licensed, semantically versioned and follows a rebase-only trunk-based policy enforced by GitHub.

Index terms – domain-driven design, clean architecture, multi-agent systems, DevOps, continuous integration, continuous delivery, predictive maintenance.

Contents

1 Introduction **3**

2 Domain-Driven Design **4**

2.1 Requirements 4

2.2 Knowledge crunching and ubiquitous language 4

2.3 Bounded contexts and context map 5

2.4 Tactical design 6

2.5 Clean Architecture 7

3 DevOps **8**

3.1 Version control and branching 8

3.2 Versioning, releases and licence 8

3.3 Build automation 8

3.4 Continuous integration 8

3.5 Quality assurance 9

3.6 Persistence 10

3.7 Containerisation and orchestration 10

3.8 Continuous delivery 10

3.9 Dependency management 10

4 Verification evidence **10**

5 Limitations and gap against the proposal **12**

6 Use of generative AI **12**

7 Conclusion and next steps **13**

A Repository map **13**

B How to run and test **13**

C Course checklist **14**

References **15**

1 Introduction

Context. The project extends a multi-agent system built by the group in four other courses of the Intelligent Embedded Systems major: Intelligent Systems Engineering (93669), Machine Learning & Data Mining (95631), Distributed Systems (87474) and Embedded Systems & IoT (77780). For Software Process Engineering the focus is strictly on DDD and DevOps, not on the agent or machine-learning internals.

The system. Three vehicle agents, a fleet coordinator and a service centre, written in Jason, predict and book maintenance. They coordinate through a shared load signal, *booking pressure*, instead of negotiating one to one, and every accepted job produces a service record that another organisation endorses. Around the agents sit an IoT control unit that receives signed telemetry from ESP32/Arduino devices over MQTT, a Weka-based prediction service, PostgreSQL, and a web dashboard (Figure 1).

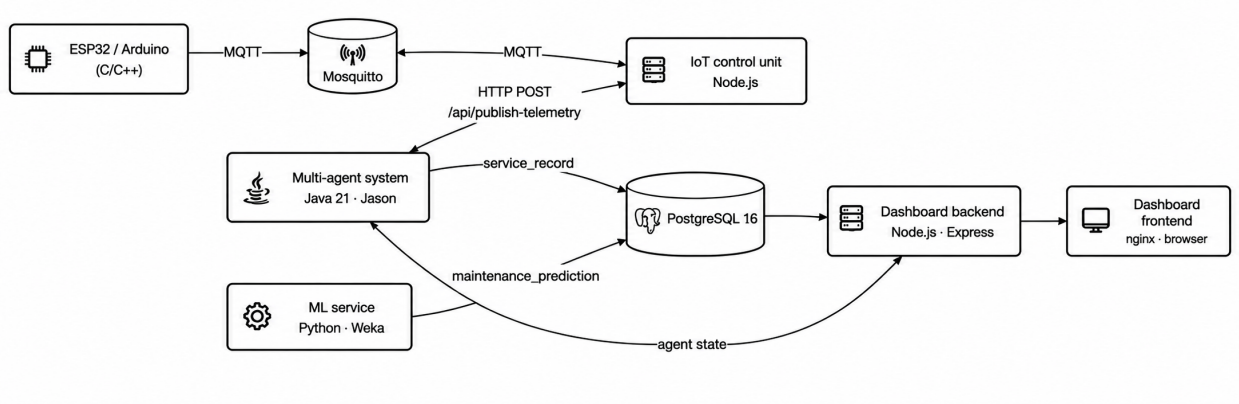


Figure 1: Runtime view: services and how they exchange data.

Platforms. Each platform lives in its own top-level folder and builds with its own standard tool. The largest platform has about half of the code, below the 75 % limit of the course checklist (physical lines of tracked source, tests included; libraries, the Weka jar and data files excluded):

Platform	Folder and build tool	Lines	Share
JVM (Java 21, Jason)	`Multi-Agent-System` – Gradle wrapper	2 973	51.5 %
Node.js / web	`Embedded-Systems-gs-main`, `MAS_system_node_server_on-main` – npm	1 895	32.8 %
Embedded C/C++	ESP32 and Arduino sketches – Arduino IDE	586	10.1 %
Python	`ML-model` – pip	323	5.6 %

Proposal versus repository. Three technologies were narrowed for the prototype: the ledger is reached through an HTTP adapter instead of a Hyperledger Fabric network, the dashboard is plain HTML and JavaScript instead of React, and the prediction model is Weka (AdaBoostM1) instead of scikit-learn. CI/CD runs on CircleCI instead of GitHub Actions. PostgreSQL, Docker and the three language platforms are as proposed.

2 Domain-Driven Design

The full domain analysis is in the repository under `docs/ddd/` (knowledge crunching, ubiquitous language, use cases, bounded contexts, tactical model). This chapter summarises it and says, for each part, whether it is **built** or a **target design**.

2.1 Requirements

ID	Requirement (from the proposal)	Realised by
FR1	A vehicle whose health is flagged shall request maintenance.	vehicle agent (urgency $\geq$ high $\rightarrow$ `book_request`)
FR2	The coordinator shall route and regulate requests across service centres.	booking pressure with evaporation; one service centre today
FR3	A service centre shall accept, defer or decline a request based on capacity, qualification and parts.	service centre agent (six decision plans)
FR4	A service centre shall log an endorsed, immutable service record.	`LogServiceRecord`, `EndorseServiceRecord`, PostgreSQL
FR5	Stakeholders shall retrieve a vehicle's service history.	dashboard table of stored records (per-VIN view still open)
FR6	Operators shall see the fleet state on a dashboard.	dashboard (agents, predictions, records)
NFR1	Deployable as containers and exercised end to end through pipelines.	Docker Compose, CircleCI delivery job
NFR2	One command to build and test each platform; the same in CI.	`make check`
NFR3	Business rules independent of Jason, HTTP, JSON and SQL.	Clean Architecture (§2.5)
NFR4	Domain and use cases testable without agents, network or broker.	hand-written fakes for every port
NFR5	A service record is decided exactly once.	aggregate invariant and database trigger
NFR6	At least two target platforms, clearly partitioned.	JVM, Python, Node.js; amd64 and arm64 images

A requirement is *done* when it is merged to `main` through a reviewed pull request, automated tests cover it where practical and pass in the pipeline, and the documentation describes it.

2.2 Knowledge crunching and ubiquitous language

The domain was explored by the three members, each the "domain expert" for the agent they had built: Dias the fleet coordinator, Danial the vehicle, Mary Anne the service centre. The inputs were the agent design, the 14-message protocol and the running scenarios. Big-picture Event Storming produced a timeline of domain events (telemetry received, anomaly detected, maintenance requested, booking accepted/deferred/declined, service completed, record endorsed) and nine *hot spots*: behaviours the team could not justify. The most important are:

- **H1** – the service record is created when a booking is *accepted*, not when the job is *completed* (still open);

- **H2** – `booking_confirmed` meant two different things (to the vehicle and to the coordinator); the language now distinguishes *booking accepted* from *capacity consumed*;
- **H4** – the coordinator always approves an endorsement, so endorsement needs an explicit rule;
- **H9** – booking pressure never evaporated in a running system (the `tick` percept was never removed, so the coordinator's `+tick` plan fired once); found while checking the stored service records, and **fixed**.

The **ubiquitous language** fixes one meaning per term and context and is used in discussion, agent code, Java code and documents. Core terms: *VIN*, *telemetry*, *anomaly*, *urgency* (low < medium < high < critical), *booking pressure*, *evaporation*, *self-deferral*, *capacity*, *slot*, *technician*, *part*, *accept / defer / decline*, *service record*, *endorsement*, *committed*, *integrity hold*, *maintenance prediction*. Words that meant different things in different contexts (for example *capacity* as advertised by a centre versus its real free slots) are listed per context.

2.3 Bounded contexts and context map

Context	Type	Responsibility	Realised in
Service & Booking	core	decide on bookings with finite capacity, technicians, parts and slots	`service_center_agent.asl`
Fleet Coordination	core	regulate demand with booking pressure, route requests, detect fleet anomaly patterns	`fleet_coordinator_agent.asl`, `BookingPressure`
Vehicle Telemetry & Health	supporting	collect and authenticate telemetry, assess health and urgency	`vehicle_agent.asl`, ESP32/Arduino, IoT control unit
Maintenance Prediction	supporting	predict maintenance need from vehicle data	`ML-model` (Weka), table `maintenance_prediction`
Trust & Service Ledger	supporting	immutable, endorsed service records and their history	`ServiceRecord` and use cases, table `service_record`

The two core contexts hold the decisions that make the system different from a fleet tracker: self-regulating demand without a central scheduler, and never accepting work that cannot be completed. They evolve together through the agent message protocol, so their relationship is a **partnership**. The supporting contexts are kept at a distance by **anti-corruption layers**, which in code are ports with adapters behind them (Figure 2): the agents see a `MaintenanceInsight`, never Weka's class labels, and a `ServiceRecord`, never SQL or a ledger transaction. Each context owns its own data: the ledger writes `service_record` through its repository port, the prediction service writes `maintenance_prediction` directly, and the dashboard only reads.

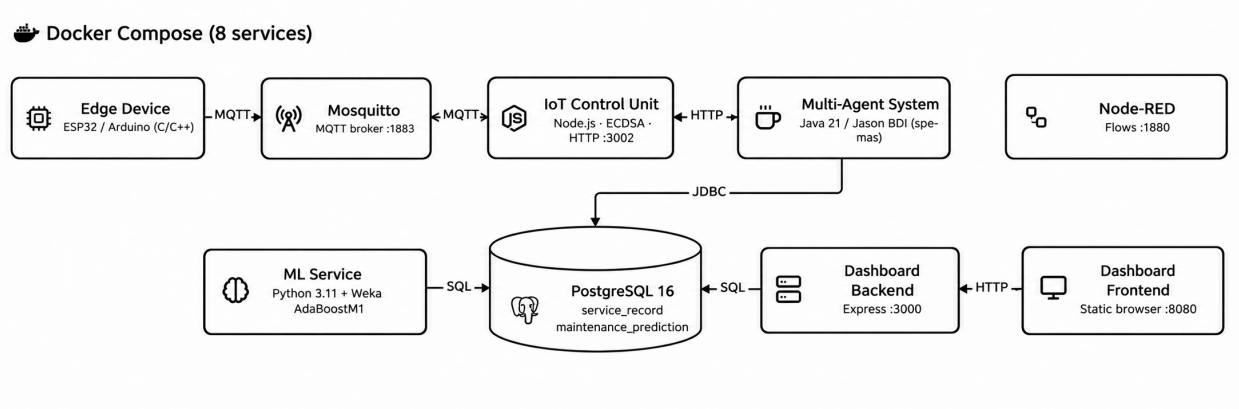


Figure 2: Context map. Arrows point from upstream to downstream; labels give the pattern and what crosses.

2.4 Tactical design

Building block	Implementation	Notes
Aggregate root	`ServiceRecord` (id, vehicle, part, technician, cost, endorsement)	compared by identity; validates every field; `restore(...)` rebuilds a stored record
Value objects	`Endorsement`, `BookingPressure`, `MaintenanceInsight`	immutable; `BookingPressure` escalates, decays and stops at both ends
Domain events	`ServiceRecordEndorsed`, `FleetAnomalyDetected`	raised by the aggregate or the use case
Repository	port `ServiceRecordRepository`; adapters `PostgresServiceRecordRepository` and `InMemoryServiceRecordRepository`	whole aggregates by id; PostgreSQL when `DATABASE_URL` is set
Application services	`LogServiceRecord`, `EndorseServiceRecord`, `LogFleetAnomaly`, `RegisterVehicle`	orchestrate domain objects and ports; no I/O of their own
Factories	`Endorsement.decide`, `BookingPressure.of/fromAtom`, `ServiceRecord.restore`	creation is simple, so static factories are enough
Domain services	none	no rule spans more than one aggregate yet

The invariant that justifies the model. In the agent code an endorsement is only a message, and nothing stopped a second message from silently overwriting the first decision. In the model the rule is explicit:

```
public ServiceRecordEndorsed endorse(String endorserId, boolean approved) {
    if (this.endorsement != null) {
        throw new IllegalStateException("Service record #" + id + " was already decided
    ");
    }
    this.endorsement = Endorsement.decide(endorserId, approved);
```

```
    return new ServiceRecordEndorsed(id, vehicleId, approved);
}
```

Since the agents now run through the use cases, the rule holds at runtime too: the service centre's `writeServiceRecord(RID, Vehicle, Part, Tech, Cost)` action runs `LogServiceRecord`, and the endorsement reply runs `EndorseServiceRecord`. PostgreSQL enforces it a second time with a trigger that refuses to update a decided row, so even a faulty adapter cannot change a committed record.

Target design for the core contexts. `docs/ddd/05-tactical-model.md` models the two core contexts as aggregates of their own: `ServiceCenter` (with `Booking`, `Technician`, `SlotCalendar`, `Capacity`, `PartsInventory` and events `BookingAccepted/Deferred/Declined`) and `FleetCoordinator` (with `MaintenanceRequest`, `BookingPressure` and `FleetAnomalyPatternDetected`), each with its invariants – for example, accepting a booking reserves slot, capacity, part and technician together or not at all.

Deliberately not modelled. There is no `Vehicle` aggregate (the agents keep only a fleet size and per-vehicle beliefs) and no `DigitalTwin` type (`readDigitalTwin` returns raw text). The agents do not yet read the stored Weka predictions: `MLPredictionPort` is still served by `MockMLPipeline`.

2.5 Clean Architecture

The Java side follows Clean Architecture (equivalently, ports and adapters): every dependency points toward the domain, and the domain imports nothing outside itself (Figure 3).

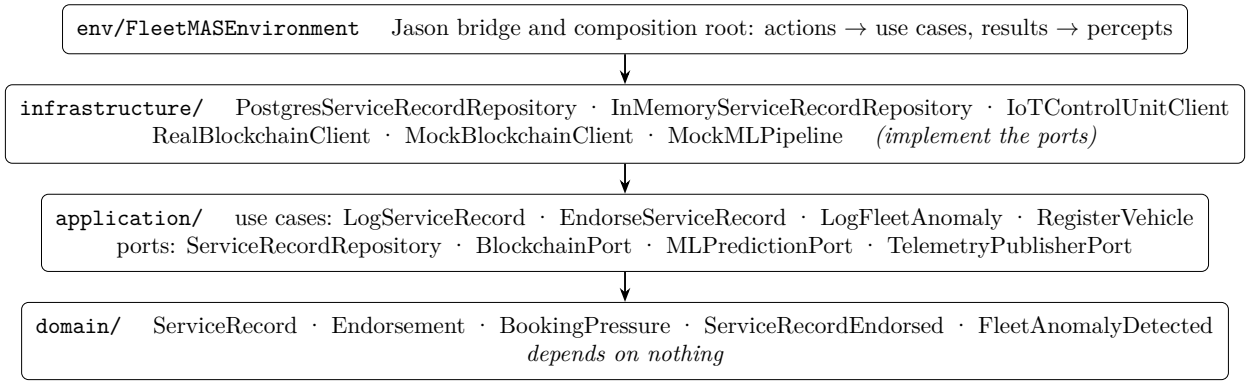


Figure 3: Layers of the multi-agent system. Arrows show the allowed direction of dependencies.

`FleetMASEnvironment.init()` is the **composition root**: the only place that creates concrete adapters and connects them to ports (for example, the PostgreSQL repository when `DATABASE_URL` is set, the in-memory one otherwise). After that it only translates Jason actions into use-case calls and results into percepts, so every use case can be tested with a hand-written fake and no Jason, network or database.

Compared with the starting point (commit `72ed91a`: one 570-line environment file mixing action dispatch, business decisions and HTTP/JSON, no automated tests), business rules are now explicit in `domain/`, external systems sit behind four ports, and the environment is a thin adapter. Known defects of the original code were kept and documented rather than silently changed (for example, the mock/real switch has no effect for the ledger, and `logFleetAnomaly` issues a `GET` instead of a `POST`).

3 DevOps

3.1 Version control and branching

Trunk-based development, as set out in the proposal: `main` always builds and passes the pipeline, work happens on short-lived branches named by type (`feat/`, `fix/`, `ci/`, `docs/`, `chore/`), and every change reaches `main` through a pull request with a checklist template. Commit messages follow Conventional Commits. The **rebasing policy is enforced by GitHub**, not left to habit: `main` requires a pull request, the four CircleCI check jobs and a linear history; merge commits are disabled, so pull requests are rebased or squashed onto `main`. Contributors rebase onto `main` and force-push only their own branches, with `-force-with-lease`. The policy is written down in `CONTRIBUTING.md`.

3.2 Versioning, releases and licence

The repository has one version, kept in `VERSION` and following Semantic Versioning; `build.gradle` and every `package.json` carry the same number, and changes are recorded in `CHANGELOG.md` (Keep a Changelog). `scripts/check-version.sh` runs first in the pipeline and fails if any component disagrees with `VERSION`, if the changelog has no section for it, or, on a release tag, if the tag disagrees. A release is a pull request that raises the version and closes the changelog section, a merge, a tag on that commit and a GitHub Release. Releases `v0.1.0` (licence, versioning, policy) and `v0.2.0` (build automation, quality tools, delivery) were made this way.

The code is released under the **MIT licence**: short, permissive and suited to an academic project others may reuse. It covers the group's own code; dependencies keep their licences. Every `package.json` declares it and every published image carries it as a label.

3.3 Build automation

Each platform builds with its own standard tool: the Gradle wrapper (`./gradlew`) for the multi-agent system, npm with lock files for the Node services, pip with `requirements.txt` for the prediction service. A root `Makefile` puts them behind one entry point: `make install`, `make check` (exactly what the pipeline runs on a pull request: version check, lint and tests on all platforms), `make up` (build and start the whole system) and `make help`.

3.4 Continuous integration

CircleCI (`.circleci/config.yml`) starts a pipeline on every push, pull request and release tag (Figure 4). Four jobs run in parallel, each in a fresh container, and report to GitHub as required status checks:

- **check-version** – the version check above;
- **test-and-build-java** – Checkstyle, JUnit 5 and JaCoCo through `./gradlew check`, with a PostgreSQL 16 container next to the build so the repository tests run against a real database;
- **test-and-lint-python** – flake8, black, pytest with coverage (a Java runtime is installed for Weka);
- **test-and-lint-node** – ESLint, the Node test runner with c8 coverage.

Each job publishes test results to CircleCI's *Tests* tab and reports to the *Artifacts* tab. No step can fail silently: no command ends in `|| true`, so a lint finding or a failing test turns the job red. (An earlier configuration did hide failures this way and showed green checks while two of the three test suites did nothing; removing that was the first change.)

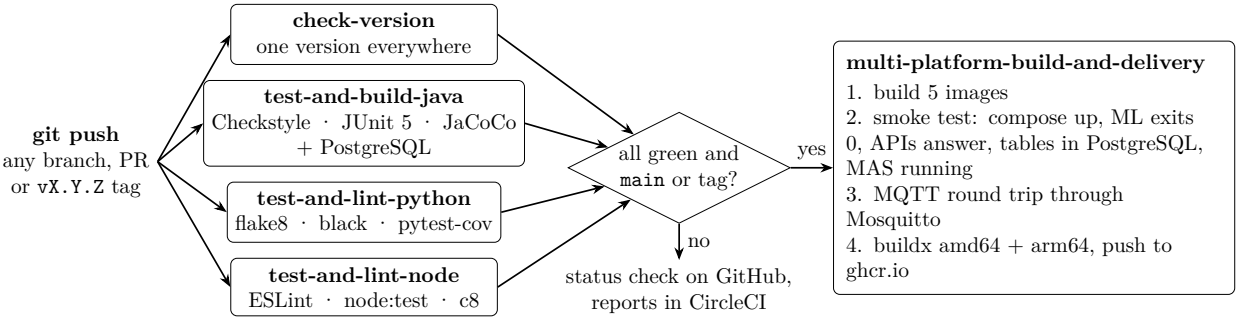


Figure 4: The CircleCI pipeline. The four check jobs run on every push; delivery runs only on `main` and release tags, after all four pass.

3.5 Quality assurance

Platform	Lint	Tests	Coverage
JVM	Checkstyle	**25** JUnit tests in 6 files: aggregate rules, three use cases with hand-written fakes, the IoT client contract, the PostgreSQL repository against a real database	JaCoCo,
Node.js	ESLint	**18** tests: syntax and dependencies, the HTTP API with no broker, and MQTT through an in-process broker	c8, 82 % lines
Python	flake8, black	**5** tests, incl. the real Weka model end to end and storing a prediction	pytest-cov,

Testing the integration points. The MAS-to-IoT connection is tested from both sides with the same payload: `IoTControlUnitClientTest` checks the Java adapter's HTTP request against a fake server, and `mqtt.test.js` runs the real control unit against an in-process MQTT broker (the payload is published on `vehicle/telemetry`, incoming telemetry is stored, `WARNING` and `CRITICAL` states are published on `vehicle/state`). The PostgreSQL repository test also proves the database refuses to change a decided record.

System test. Before publishing anything, the delivery job starts the whole system with Docker Compose and checks that the prediction service ran and exited successfully, that the IoT API, the dashboard backend and frontend answer, that the ML predictions and service records are read back from PostgreSQL, and that the multi-agent system is running. It then performs an **MQTT round trip through the real Mosquitto container**: it sends the request the Java MAS sends, waits for that message on the broker and checks that it comes back into the control unit's history. If any check fails, the job prints the containers' logs and nothing is published.

Problems the tools found. Turning the tools on found real defects, all fixed: the IoT unit never opened the Arduino serial port (an undefined variable, found by ESLint); the dashboard's fleet endpoint waited forever when agents were unreachable; the Node manifest was named `PACKAGE.JSON`, which breaks on Linux; one Java line held three statements; and, once records were stored, booking pressure turned out never to evaporate (hot spot H9). Two smoke-test checks were made robust against timing: waiting for an already-finished container, and other telemetry arriving on the broker during the MQTT round trip.

3.6 Persistence

PostgreSQL 16 runs as a container; the schema (`deploy/postgres/schema.sql`) is loaded on first start and has one table per owning context:

- `service_record` (Trust & Service Ledger) – written by the agents through `PostgresServiceRecordRepository`; a `CHECK` keeps the three endorsement columns all empty or all filled, and a trigger refuses to update a decided row;
- `maintenance_prediction` (Maintenance Prediction) – one row per Weka prediction, written by the Python service for each instance of the template (nine today).

The domain never sees SQL: the repository port hides it, and without `DATABASE_URL` the in-memory adapter is used, so unit tests and a local `gradle run` need no database. The database port is not published outside Docker, so it cannot clash with a local installation.

3.7 Containerisation and orchestration

Every service has a Dockerfile; the backend and the IoT unit run as a non-root user and install exactly the versions in their lock files. The multi-agent system's image compiles with Gradle on the build machine and copies only the jar, libraries and agent files into a small Java runtime image, so the arm64 build never runs Gradle under emulation. `docker-compose.yml` describes the whole system as **eight services**: Mosquitto, the IoT control unit, PostgreSQL, the multi-agent system, the prediction service, the dashboard backend and frontend, and Node-RED. Services find each other by name, the multi-agent system and the prediction service wait until PostgreSQL is healthy, and `docker compose pull && docker compose up -d` runs the published images.

The MQTT broker and the database use demo credentials (anonymous MQTT, a fixed PostgreSQL password), acceptable for a local demonstration but not for production; the defence against forged telemetry is the ECDSA signature on every record.

3.8 Continuous delivery

The delivery job runs on a CircleCI virtual machine with its own Docker engine, only on `main` and release tags and only after the four checks pass. After the system test it uses Docker Buildx with QEMU to build every image for **linux/amd64 and linux/arm64** and pushes it to `ghcr.io/dias266/spe-<service>`, tagged with the commit (`sha-` plus seven characters) or the version, and `latest`. Images carry labels for source, version, commit and licence. The registry token is a CircleCI project secret, never in the repository and masked in logs; if it is missing the job stops with a clear message. Pull requests from forks get no secrets and never run delivery.

3.9 Dependency management

Dependencies are declared in `build.gradle`, in the `package.json` files with lock files, and in `requirements.txt` / `requirements-dev.txt`; the pipeline installs from these files only. Installed packages, editor caches and OS files are excluded by `.gitignore` (the earlier repository had about 3 400 such files committed). Automatic updates (for example Dependabot, opening pull requests that go through the same pipeline) are planned.

4 Verification evidence

Pipeline.

Service records written by the running agents. After `docker compose up`, the agents' bookings are stored and endorsed:

```
$ docker compose exec postgres psql -U fleet -d fleet -P pager=off \
    -c "SELECT id, vehicle_id, part, technician, cost, endorser_id, approved FROM service_record ORDER BY
     id;"
 id |   vehicle_id   |    part    | technician | cost |       endorser_id       | approved
----+----------------+------------+------------+------+-------------------------+----------
  1 | vehicle_agent3 | oil_filter | tech_a     |   85 | fleet_coordinator_agent | t
  2 | vehicle_agent2 | oil_filter | tech_a     |   85 | fleet_coordinator_agent | t
  3 | vehicle_agent1 | oil_filter | tech_a     |   85 | fleet_coordinator_agent | t
```

The cost is the part price plus one labour hour (an oil filter: 25 + 60). A manual `UPDATE` of a decided row is refused by the database (*service record 1 was already decided*).

Dashboard. Figure 5 shows the live agent state above the two tables read from PostgreSQL: the Weka predictions with confidence and filtered decision, and the endorsed service records. The service centre's own record counter and the number of stored records agree.

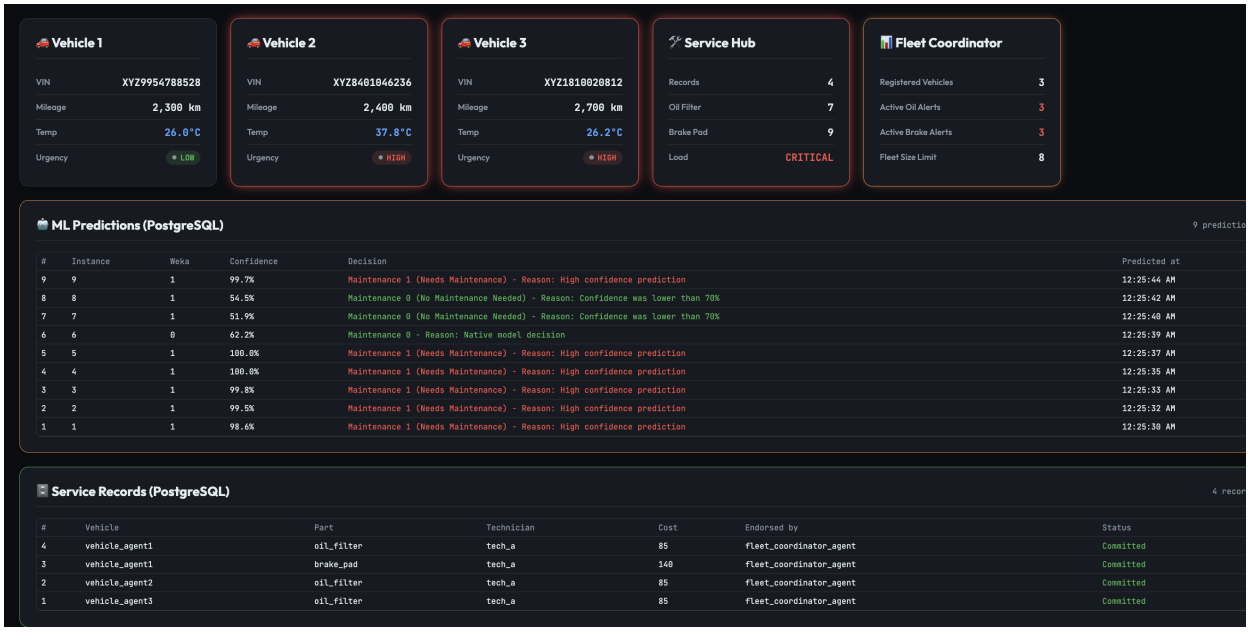


Figure 5: Fleet Operational Center: agent state, ML predictions and service records from PostgreSQL.

5 Limitations and gap against the proposal

Proposal item	Status	Note
Trunk-based development with a rebasing policy	done	enforced by branch protection
CI/CD pipelines, test automation, coverage, linters	done	on CircleCI instead of GitHub Actions
Automated delivery of container images	done	five multi-architecture images on ghcr.io
Semantic versioning, OSI licence	done	v0.1.0, v0.2.0; MIT
Persistence (PostgreSQL)	done	service records and predictions
Three platforms exercised end to end	done	smoke test and MQTT round trip; agent decisions not yet asserted automatically
Tactical patterns for the core contexts	done	designed in `docs/ddd`; the agents still decide in AgentSpeak
Hyperledger Fabric, React, scikit-learn	done	HTTP adapter, plain HTML, Weka

Other known limits: the agents use mock ML insights instead of the stored Weka predictions; the service record is logged when a booking is accepted, not when the job is completed (H1); the agent restarts its record counter at 1, so a restart on old data refuses the first booking (each demo starts with `docker compose down -v`); there is one service centre, so routing across centres is not exercised; the dashboard reads agent state by parsing the Jason inspector's HTML; and the pipeline tests on Linux only, although images are built for two architectures.

6 Use of generative AI

Item	Declaration
Tool	Claude, an AI assistant from Anthropic, in conversational and coding-agent use.
Used for	understanding the proposal and repository; proposing the layered design; writing domain, application and infrastructure code and tests on all three platforms (incl. the MQTT tests, the service-record wiring, the PostgreSQL repository and prediction store); the CircleCI configuration, Dockerfiles, compose file, Makefile, version check and linter configurations; the dashboard tables; diagnosing pipeline and runtime problems (task ordering, hidden failures, smoke-test timing, the evaporation bug); drafting the changelog, contribution guide, DDD documents and this report.
Not used for	the original agent plans, the ESP32/Arduino firmware, the prediction model and the ledger backend.
How output was checked	every change reached `main` through a pull request that had to pass the pipeline; the group ran the system locally (`gradle run`, `docker compose up`, database queries, the dashboard) and reviewed the changes; no independent code review was made.
Responsibility	the group members remain responsible for all content submitted.

7 Conclusion and next steps

Both halves of the proposal work for most of their scope. On the DevOps side every proposal item runs: a versioned, MIT-licensed repository with an enforced rebase-only policy, one-command builds, a pipeline that lints, tests and measures coverage on three platforms on every push, and automated delivery of multi-architecture images after a system test that now includes the database and the MQTT broker. On the design side there are five bounded contexts with a context map, two supporting contexts modelled and running behind ports, and a Clean Architecture refactor of the agent system whose central invariant holds at runtime and in the database.

Next steps, in order: release `v0.3.0`; connect the agents to the stored predictions through a real `MLPredictionPort` adapter; implement the core aggregates and let the agents call them; log service records on *service completed* (H1); assert agent behaviour in the system test; add automatic dependency updates.

A Repository map

Path	Purpose
`Multi-Agent-System/`	Jason agents, the layered Java code, tests, Checkstyle rules, Dockerfile, design documents
`Embedded-Systems-gs-main/`	ESP32/Arduino sketches, the Node IoT control unit with its tests and Dockerfile
`ML-model/`	Weka model, template, prediction script, tests, Dockerfile
`MAS_system_node_server_on-main/back/`	dashboard backend and frontend with their Dockerfiles
`deploy/`	Mosquitto configuration and PostgreSQL schema
`docs/ddd/`	domain analysis
`.circleci/config.yml`, `docker-compose.yml`	pipeline; the whole system as eight services
`Makefile`, `scripts/`	one-command build and test; version check
`VERSION`, `CHANGELOG.md`, `LICENSE`, `CONTRIBUTING.md`	version, history, licence, branching policy

B How to run and test

```
make install && make check        # everything the pipeline checks (Java 21, Python
    3.11, Node 20+)

docker compose down -v            # start from an empty database
docker compose up -d --build      # or: docker compose pull && docker compose up -d
# dashboard: http://localhost:8080    IoT API: http://localhost:3002
docker compose exec postgres psql -U fleet -d fleet -c "SELECT * FROM service_record;"
docker compose down -v
```

C Course checklist

Checklist area	Status	Where
Version control	done	§3.1
Versioning	done	§3.2 (HTTP routes are not versioned)
Project structure	done	§1, §3.3, §3.9, Appendix A
Licensing	done	§3.2
Quality assurance	done	§3.5
DevOps pipelines	done	§3.4, §3.7, §3.8
Domain-Driven Design	done	§2
Requirements	done	§2.1
Multi-platform	done	§1 (largest platform 51.5 %)
AI exploitation	done	§5

References

[1] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.

[2] V. Vernon, *Implementing Domain-Driven Design*. Addison-Wesley, 2013.

[3] V. Khononov, *Learning Domain-Driven Design*. O'Reilly Media, 2021.

[4] A. Brandolini, *Introducing EventStorming*. Leanpub, 2021.

[5] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

[6] A. Cockburn, "Hexagonal architecture." `https://alistair.cockburn.us/hexagonal-architecture/`, 2005. Accessed October 2026.

[7] R. H. Bordini, J. F. Hübner, and M. Wooldridge, *Programming Multi-Agent Systems in AgentSpeak using Jason*. Wiley, 2007.

[8] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, and I. H. Witten, "The WEKA data mining software: An update," *SIGKDD Explorations*, vol. 11, no. 1, pp. 10–18, 2009.

[9] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[10] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.

[11] G. Kim, J. Humble, P. Debois, and J. Willis, *The DevOps Handbook*. IT Revolution Press, 2016.

[12] M. Fowler, "Continuous integration." `https://martinfowler.com/articles/continuousIntegration.html`, 2006. Accessed October 2026.

[13] P. Hammant, "Trunk based development." `https://trunkbaseddevelopment.com`, 2017. Accessed October 2026.

[14] T. Preston-Werner, "Semantic versioning 2.0.0." `https://semver.org/spec/v2.0.0.html`. Accessed October 2026.

[15] "Conventional commits 1.0.0." `https://www.conventionalcommits.org/en/v1.0.0/`. Accessed October 2026.

[16] O. Lacan, "Keep a changelog 1.1.0." `https://keepachangelog.com/en/1.1.0/`. Accessed October 2026.

[17] Open Source Initiative, "The MIT license." `https://opensource.org/license/mit`. Accessed October 2026.

[18] CircleCI, "Circleci documentation." `https://circleci.com/docs/`. Accessed October 2026.

[19] Docker Inc., "Docker compose and docker buildx documentation." `https://docs.docker.com/`. Accessed October 2026.

[20] The PostgreSQL Global Development Group, "PostgreSQL 16 documentation." `https://www.postgresql.org/docs/16/`. Accessed October 2026.

[21] OASIS, "MQTT version 5.0." OASIS Standard, https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html, 2019.

[22] National Institute of Standards and Technology, "Digital signature standard (DSS)," FIPS 186-5, NIST, 2023.